

Chapter 1: A Tutorial Introduction

Relational graph edition. Pull-shaped. Top-down.

How to use this document

Red boxes are checkpoints. Stop and answer them in chat before continuing. Green boxes are practice problems with inline answers. Blue box is the target problem you will solve by the end. Read the graph first, then walk the edges one at a time.

Big picture

This chapter bootstraps you from zero to writing real C programs. A C program is a set of functions. One function is called `main` -- the OS calls it when your program starts. Functions call other functions, pass data via arguments, and return results. Every concept below is a tool you need to solve the target problem.

Target problem

Write a program that reads lines from stdin and prints the longest one. It must use: a main function, a `getline` function, a copy function, character arrays, and an external variable for the max length seen so far.

The Graph

Node inventory -- memorize these

Node	What it is	Why it matters
<code>main()</code>	Entry point function	OS calls this to start your program
<code>printf(fmt, ...)</code>	Formatted output to stdout	Primary way to print results
<code>getchar()</code>	Read one character from stdin	Returns int (char value or EOF)
<code>putchar(c)</code>	Write one character to stdout	Outputs a single byte
EOF	End-of-file sentinel (usually -1)	Signals no more input
int, float, char	Basic data types	int: integers, float: decimals, char: single bytes
for / while	Loop constructs	Repeat a block until condition is false

#define NAME val	Symbolic constant	Preprocessor replaces NAME with val everywhere
type name[SIZE]	Array declaration	Contiguous block of SIZE elements
return-type fn(params)	Function definition	Named, reusable block of code
extern	External variable declaration	Variable defined in another file/scope

Edge inventory -- how nodes connect

Edge	From	To	Rule
E1	main()	printf()	main calls printf to produce output
E2	variables	printf()	Format specifiers (%d, %f) insert variable values
E3	for/while	statements	Loop body executes while condition is true
E4	#define	any code	Preprocessor substitutes name with value before compilation
E5	getchar()	variable	Returns next char (as int); EOF when input ends
E6	array[i]	value	Index i selects one element from the block
E7	function()	return value	Function executes body, returns a value to caller
E8	extern	variable	Declares a variable defined elsewhere is accessible here

Zoom in -- one edge at a time

Edge 1: main() calls printf()

Every C program must have exactly one function called `main`. The OS calls it. `printf` is a library function that writes formatted text to `stdout`. The `\n` in the format string is a newline character.

```
#include <stdio.h>

int main(void) {
    printf("hello, world\n");
    return 0;
}
```

Problem 1.1: Write a program that prints your name on one line and your age on the next line.

Answer: `printf("Alice\n"); printf("Age: 30\n");` inside `main`. Two `printf` calls, each ending with `\n`.

Edge 2: Variables feed into printf() via format specifiers

Declare a variable with type `name`; or type `name = value`; . In `printf`, `%d` prints an int, `%f` prints a float, `%c` prints a char.

```
int fahr = 100;
float celsius = (5.0/9.0) * (fahr - 32);
printf("%d F = %.1f C\n", fahr, celsius);
```

Key rule: Integer division truncates. 5/9 is 0. Write 5.0/9.0 to get floating-point division.

Problem 1.2: Write a program that converts 212 F to Celsius and prints the result with one decimal place.

Answer: `float c = (5.0/9.0)*(212-32); printf("%.1f\n", c);` Output: 100.0

Edge 3: for/while loops repeat a block

`for (init; test; increment) { body }` -- `init` runs once, then: `test`, `body`, `increment`, repeat. Equivalent to `while` but keeps loop machinery on one line.

```
for (int fahr = 0; fahr <= 300; fahr += 20) {
    printf("%3d %6.1f\n", fahr, (5.0/9.0)*(fahr-32));
}
```

Problem 1.3: Write a `for` loop that prints the numbers 1 through 10, one per line.

Answer: `for (int i = 1; i <= 10; i++) printf("%d\n", i);`

Edge 4: #define replaces magic numbers with names

The preprocessor replaces every occurrence of the name with the value before the compiler sees it. No semicolon. No equals sign. Just `#define NAME value`.

```
#define LOWER 0
#define UPPER 300
#define STEP 20
for (int f = LOWER; f <= UPPER; f += STEP)
```

Problem 1.4: Replace the magic numbers 1 and 10 in your Problem 1.3 answer with `#define` constants.

Answer: `#define START 1` and `#define END 10`, then use them in the for loop.

CHECKPOINT A

Before continuing, write a complete program (with `#include`, `main`, and a for loop) that prints a Celsius-to-Fahrenheit table from 0 to 100 in steps of 10. Use `#define` for the bounds. Report your code in chat.

Edge 5: getchar()/putchar() -- character-at-a-time I/O

`getchar()` returns the next character from `stdin` as an `int`. When input ends, it returns `EOF` (defined in `stdio.h`, usually `-1`). Must store in `int`, not `char`, because `EOF` must be distinguishable from any valid `char` value.

```
int c;
while ((c = getchar()) != EOF)
    putchar(c);
```

This copies `stdin` to `stdout`, one character at a time, until `EOF`.

Problem 1.5: Write a program that counts the number of characters in its input.

Answer: `long count = 0; int c; while ((c = getchar()) != EOF) ++count; printf("%ld\n", count);`

Edge 6: Arrays -- indexed blocks of same-type values

`int counts[10];` declares 10 ints, indexed 0 through 9. Array indices start at 0. Accessing beyond bounds is undefined behavior (no safety net).

```
int digit_count[10];
for (int i = 0; i < 10; i++)
    digit_count[i] = 0;
int c;
while ((c = getchar()) != EOF)
    if (c >= '0' && c <= '9')
        ++digit_count[c - '0'];
```

Key trick: `c - '0'` converts a digit character to its numeric value. `'5' - '0' = 5`. Works because digit characters are contiguous in ASCII.

Problem 1.6: Write a program that counts how many times each lowercase vowel (a, e, i, o, u) appears in its input.

Answer: Use an array of 5 ints. Use `if/else-if` or `switch` on each `char` to decide which counter to increment.

CHECKPOINT B

Write a word-counting program from scratch. A word is any sequence of characters that does not contain a space, tab, or newline. Count words, lines, and characters. Report your code in chat.

Edge 7: Functions -- named reusable blocks

A function has: return type, name, parameter list, body. Call it by name with arguments. It returns a value (or void for no value). Functions must be declared before use (prototype or definition).

```
int power(int base, int exp) {
    int result = 1;
    for (int i = 0; i < exp; i++)
        result *= base;
    return result;
}

/* In main: */
printf("2^10 = %d\n", power(2, 10));
```

Problem 1.7: Write a function `int max(int a, int b)` that returns the larger of two integers. Call it from main.

Answer: `int max(int a, int b) { if (a > b) return a; return b; }`

Edge 8: Call by value -- functions get copies

C passes all arguments by value. The function receives a copy of each argument. Modifying the copy inside the function does NOT change the original in the caller. This is different from arrays -- when you pass an array name, you pass the address of element 0.

Problem 1.8: If you write `void inc(int x) { x++; }` and call `inc(n)`, does `n` change?

Answer: No. The function increments its local copy. `n` is unchanged.

Edge 9: Character arrays -- strings end with \0

A string in C is a character array terminated by `'\0'` (null character, value 0). String literal `"hello"` is stored as: `h, e, l, l, o, \0` -- 6 bytes total. The `\0` tells functions like `printf` where the string ends.

```
int getline(char s[], int lim) {
    int c, i;
    for (i = 0; i < lim-1 && (c=getchar()) != EOF && c != '\n'; i++)
        s[i] = c;
    if (c == '\n') s[i++] = c;
    s[i] = '\0';
    return i;
}
```

Problem 1.9: Write a function `void copy(char to[], char from[])` that copies string from into to, including the `\0`.

Answer: `int i = 0; while ((to[i] = from[i]) != '\0') i++;`

Edge 10: extern -- sharing variables across functions

A variable defined outside all functions is an external (global) variable. Any function in the same file can access it by name. Functions in OTHER files use `extern int varname;` to declare access. External variables persist for the entire program lifetime. Use sparingly -- they create hidden dependencies.

Problem 1.10: What is the difference between a definition and a declaration of an external variable?

Answer: Definition allocates storage: `int max_len;` (outside functions). Declaration announces existence: `extern int max_len;` (no storage created).

Solving the target problem

Walk: E9 (getline with char array) + E7 (functions) + E10 (extern) + E6 (array comparison)

```
#include <stdio.h>
#define MAXLINE 1000

int my_getline(char line[], int maxline);
void copy(char to[], char from[]);

int main(void) {
    int len, max;
    char line[MAXLINE], longest[MAXLINE];

    max = 0;
    while ((len = my_getline(line, MAXLINE)) > 0)
        if (len > max) {
            max = len;
            copy(longest, line);
        }
    if (max > 0)
        printf("%s", longest);
    return 0;
}
```

Edges walked: E4 (#define MAXLINE), E6 (char arrays line[] and longest[]), E7 (functions my_getline and copy), E3 (while loop over input lines), E9 (string termination with \0 in getline), E2 (printf to output the result).

CHECKPOINT C

Modify the longest-line program to print ALL lines longer than 80 characters (not just the longest). Report your modified main() in chat.

Final exam

Problem 1: Write a program that prints all input lines in reverse order (last character first).

Problem 2: Write a function that removes trailing blanks and tabs from a string.

Problem 3: Write a program that replaces each tab with the right number of spaces to reach the next tab stop (every 8 columns).

Problem 4: Write a program that folds long input lines into two or more shorter lines after the last blank before column 80.

Problem 5: Rewrite the temperature conversion program using a function for the conversion formula.

CHECKPOINT D

Complete at least 3 of the 5 problems above. Report your solutions in chat.

Answer key

- 1. Reverse lines:** Walk E5 + E9: read into char array with `getline`, then loop from `len-1` down to 0 calling `putchar`.
- 2. Remove trailing blanks:** Walk E9 + E6: find the `\0`, scan backwards past spaces/tabs, place new `\0`.
- 3. Tab expansion:** Walk E5 + E3: track column count, on tab output spaces until `column % 8 == 0`.
- 4. Line folding:** Walk E5 + E6 + E9: buffer line in array, track column, when near 80 scan back for last blank.
- 5. Conversion function:** Walk E7 + E3: define `float celsius(int fahr)`, call inside a for loop with `printf`.